

John Cheek
Alexander Card
CSC 481
10 November 2025

Milestone 5 Reflection

In this milestone, implementing the required engine features was probably the most straightforward this time around. Our entity creation 'factory' class made the memory management implementation rather straightforward, whilst creating input chords was facilitated by our input system. Instead of working on two different singleplayer games, we spent most of our time working together as a team to finally implement the card game we have been talking about all semester. This was a huge task that was definitely more than sufficient for this class, but it was a lot of fun and I learned a lot about how engine systems interact with and can facilitate the creation of game mechanics.

For memory management, we created a pool allocator and a game object allocator, each with support for generic types. The pool allocator allocates enough space for a given number of entities for a given entity type. In order to implement this properly, we needed to include soft caps for each type of entity (but those can be changed easily). The pool allocator contains pointers to everywhere in memory where the memory for game objects are stored. The game object allocator uses the corresponding pool allocator (taking it as a parameter) and allows the user to create game objects with it. The users use the `GameObjectAllocator` to create their desired game object, while the `PoolAllocator` takes that object, creates it in memory, and stores a reference to it. Figure 1 shows the pool allocator and game object allocators being instantiated for each entity type, while Figure 2 shows the entity system using these memory allocators to create platform game objects (note that these specific figures are for the platformer based demo game we used to test our team engine features). To account for these changes, our `EntityManager` had to be updated to now take both the entity created with the `GameObjectAllocator` and the corresponding `PoolAllocator`. This change was made so that we can still keep our entity manager's functionality, but have it use the new memory management features. Because of this change, the entity manager can still be used to both create and delete entities through the entity manager, but still using the new engine memory management tools.

Input chords were a rather straightforward addition. We used our input system developed way back in MS1 to determine what the last key pressed was and what the current key pressed is. Since the input chords we added are a short two key sequence, we just need to check these two inputs. If the case meeting these two inputs is met, an input event will be raised. The event system we created in the last milestone works well with input events, as a chord will allow a special input event to be raised, so the event manager will make sure that special input event is addressed as input events have high priority. Additionally, we did need to create a special case in our input system to process the special chord event. As an example of the input chords working,

double tapping A or D will raise a dash event, this dash event has its own special type within Input.cpp. Figure 3 shows the input event being detected and subsequently raised.

These two features were a natural addition to the engine features we have been working on all semester and implementing them was rather straight forward and relatively simple compared to the other work we've done in the class. While these were intentionally supposed to be easier team engine features (thanks by the way) I also think their ease to implement was in part due to the flexible engine features and systems we have been working on all semester. Throughout the course, our team's main design philosophy was creating an efficient design, where the various engine systems all work together effectively in a way that makes sense, and creating a modular design, where you could easily develop new features that use existing modular, flexible, and extensible engine systems to facilitate their implementation. I am overall very proud of how our team's engine achieves these goals. The entity component system was the most reusable part of our code that we have used in every milestone since MS3 and came in very handy when making 'Lizard 101' (more on that later). The biggest weakness of our engine design was the networking functionality (and I am very thankful we did not have to use that for the game part of this milestone). While I am glad to have gotten some basic experience with networking in this course; our engine's network model was serviceable for the simple projects in this course, but I can imagine it being much more complicated and causing lots of issues in larger scale projects like 'Lizard 101.' Our ECS did facilitate network implementation, but we ran into roadblocks when sending events over the network like I talked about in MS4. I am sure there are better ways to implement network systems but as none of our group members had much experience with networking, I think that shows in our communication pattern and as such, client-server communication is one of the biggest weaknesses of our engine design.

If we could start over and develop our game engine all over again, I'd build the entity-component system game object model and event system *first*, then build up the other systems (like physics, input, collisions, etc) around that. Because of the way the course was structured, we already had many core engine systems like physics, input, and collisions, implemented *before* we had the new game object model or events. This made implementing new and larger features like networking more complicated, as each larger feature would require a significant refactor of the engine code. Even after the event system and ECS was added, there were still parts of the engine using previous systems so we were not able to fully benefit from having our engine relying fully on events or our game object model. Ideally, we would have this done the other way around (building the ECS and Events first). The ECS is very extensible and reusable and events are the most effective way to handle and process events in games. If we could start with implementing those two features first, and then build up the other, more complicated engine systems around those features, we could create an overall much more efficient game engine that is easier to develop complex games in. Building the engine in this way would allow for very usable and modular systems with the ECS and event management as a robust foundation.

For the games section of this milestone, our team was very thankful to have the choice to work on one larger scale game as a team instead of the typical individual game projects. This whole semester our group has talked about making a card game specific engine and while the requirements for the milestones kind of limited that and caused us to focus more on platformer like games for our individual project, we decided to work as a team to create a simple card game using our game engine and so we created 'Lizard 101' using the paper card game of the same name designed by one of our team members his freshman year. Creating this in our engine was a very difficult, but certainly a rewarding challenge. I will acknowledge that we didn't end up using all of our engine features in Lizard 101 because those features had been designed for platformer games rather than our card game. Ideally we would have created new engine features in our engine for use in the game, and then program the game based off of those, but due to time and requirement constraints, we strayed away from doing that. However, our engine was still used a great deal in the creation of the game. The timeline is used to handle events, the event manager is responsible for the collision and input events used in the game, and of course the Game Object Model is responsible for every entity that's seen in the game. Figure 4 shows a screenshot of the main gameplay. In this screenshot, each texture on the screen represents a game object. This includes the background, enemies, players, cards, and status symbols. The components of these game objects vary by their function. The background simply has a TextureRenderer component, while the players and cards both have a TextureRender, BoxCollider. Figure 5 shows the enemy entities being created. Here you can see it uses a tag for classification, a textureRender for the sprite, and a boxCollider tag for collisions. The box collider tag is used so that cards that target an enemy can interact with it. Figure 6 shows an input event being raised whenever the left mouse button is pressed, which is how the player primarily interacts with the game. These flexible systems we designed helped streamline the development of Lizard101 as any game in the engine will use those systems repeatedly. Their use in this project is a testament of their flexibility and how those systems can be adapted for any type of game. However, if we were to do it all over again and had more time to design, we would add a component type which we could add to a game object that is directly responsible for the card's logic and would relegate all card specific behavior (like the effects of the card and the scope/target of the card) to the component. Currently the card's effects are implemented as a struct which is created at runtime and given to the card while in hand and their scope is determined when collisions between cards are processed. Using a component for this rather than having a bunch of C++ source code scattered throughout the code would make adding new cards more straightforward and less tedious.

As we only made the one game for this milestone, and that most of the logic for the game is outside of main, I am unable to run the diff command and gift many fruitful results, so instead I will run the diff command on the main.cpp file used in MS5 and the main.cpp file used in MS3. Both of these files have a large amount of game specific logic located in main as milestone 3 was a singleplayer game as well (MS 4, while more recent, was a multiplayer game so most of the logic is in client.cpp and server.cpp). Between these two files there were 224 lines in common,

1017 added, and 130 removed. The parts the two files had in common were the SDL setup and initialization, the use of the game timeline, physics system, and event manager (which Lizard101 passes to Lizard101Controller.cpp), and some entity creation logic. This is to be expected as Lizard101 is a significantly more complex game, and the bulk of its logic and entity setup is elsewhere. Regardless, the lines in common show that both games used the same core engine systems we have been developing all semester.

Overall, our goal was to create a modular and easily expandable game engine that would (eventually) support a card game and I'm very proud of how our team accomplished that. Throughout all the other milestones, the process of making games was streamlined by the use of our engine features. While Lizard101 was a significant change from the rest of the class, we still were able to use many of our core engine systems without the need for much (or any) change to those systems. I am very proud of what we accomplished when making that game, especially in such a short amount of time. My main goals for this course were to get more hands-on experience with C++ (as it is pretty much the standard language for game development) and gain a stronger understanding of how to create engine systems and how those engine systems interact with gameplay. I certainly got more experience with C++, both through creating engine systems and just straight up coding gameplay in C++, and through that development I was able to gain more of an understanding and appreciation for how game engines are designed, how their features work, and how their systems support other systems and facilitate game creation. I am very excited to take the skills I learned in this course forward, and keep working on Lizard 101 in my own custom engine!

Figure 1: Pool and Game object allocators being created for the team submission

```
PoolAllocator<Player>          playerPool(8);
PoolAllocator<MovingPlatform> movingPlatformPool(32);
PoolAllocator<StationaryPlatform> stationaryPlatformPool(32);
PoolAllocator<Entity>          genericEntityPool(64);
PoolAllocator<PauseButton>     pauseEntityPool(2);

GameObjectAllocator<Player>     playerAllocator(&playerPool);
GameObjectAllocator<MovingPlatform> movingPlatformAllocator(&movingPlatformPool);
GameObjectAllocator<StationaryPlatform> stationaryPlatformAllocator(&stationaryPlatformPool);
GameObjectAllocator<Entity>     genericEntityAllocator(&genericEntityPool);
GameObjectAllocator<PauseButton> pauseButtonAllocator(&pauseEntityPool);
```

Figure 2: Platformer objects created using memory managers

```
auto* horiz = movingPlatformAllocator.create("horizontalPlatform", 650.0f, 600.0f, 96, 32, true, 100.0f, 1000.0f);
horiz->setTag("PLATFORM");
horiz->addComponent<TextureRenderer>(renderer, "media/darkworld_platform_mossystone.png");
horiz->addComponent<BoxCollider>();
if (auto* c = horiz->getComponent<BoxCollider>()) c->setEventManager(&eventManager);
manager.addEntity(horiz, makePoolDeleter(movingPlatformAllocator));

auto* vert = movingPlatformAllocator.create("verticalPlatform", 250.0f, 650.0f, 96, 32, false, 150.0f, 400.0f);
vert->setTag("PLATFORM");
vert->addComponent<TextureRenderer>(renderer, "media/darkworld_platform_mossystone.png");
vert->addComponent<BoxCollider>();
if (auto* c = vert->getComponent<BoxCollider>()) c->setEventManager(&eventManager);
manager.addEntity(vert, makePoolDeleter(movingPlatformAllocator));

auto* plat1 = stationaryPlatformAllocator.create("Platform1", 1770.0f, 540.0f, 96, 32);
plat1->setTag("GROUND");
plat1->addComponent<TextureRenderer>(renderer, "media/darkworld_platform_mossystone.png");
plat1->addComponent<BoxCollider>();
if (auto* c = plat1->getComponent<BoxCollider>()) c->setEventManager(&eventManager);
manager.addEntity(plat1, makePoolDeleter(stationaryPlatformAllocator));
```

Figure 3: Input system chord detection and raising chord events

```
// Dash chord state: Jump + A/D
static bool prevDashLeft = false;
static bool prevDashRight = false;

const bool curDashLeft = curShift && curA;
const bool curDashRight = curShift && curD;

if (curA != prevA) {
    raiseInput(InputAction::Kind::MoveLeft, curA);
    EVENT_LOG(std::string("[raise] Input MoveLeft pressed=") + (curA ? "1" : "0"));
    prevA = curA;
}
if (curD != prevD) {
    raiseInput(InputAction::Kind::MoveRight, curD);
    EVENT_LOG(std::string("[raise] Input MoveRight pressed=") + (curD ? "1" : "0"));
    prevD = curD;
}
if (curJump != prevJump) {
    raiseInput(InputAction::Kind::Jump, curJump);
    EVENT_LOG(std::string("[raise] Input Jump pressed=") + (curJump ? "1" : "0"));
    prevJump = curJump;
}

if (curDashLeft != prevDashLeft) {
    raiseInput(InputAction::Kind::DashLeft, curDashLeft);
    prevDashLeft = curDashLeft;
}

if (curDashRight != prevDashRight) {
    raiseInput(InputAction::Kind::DashRight, curDashRight);
    prevDashRight = curDashRight;
}
}
```

Figure 4: Lizard 101 Gameplay featuring many game objects

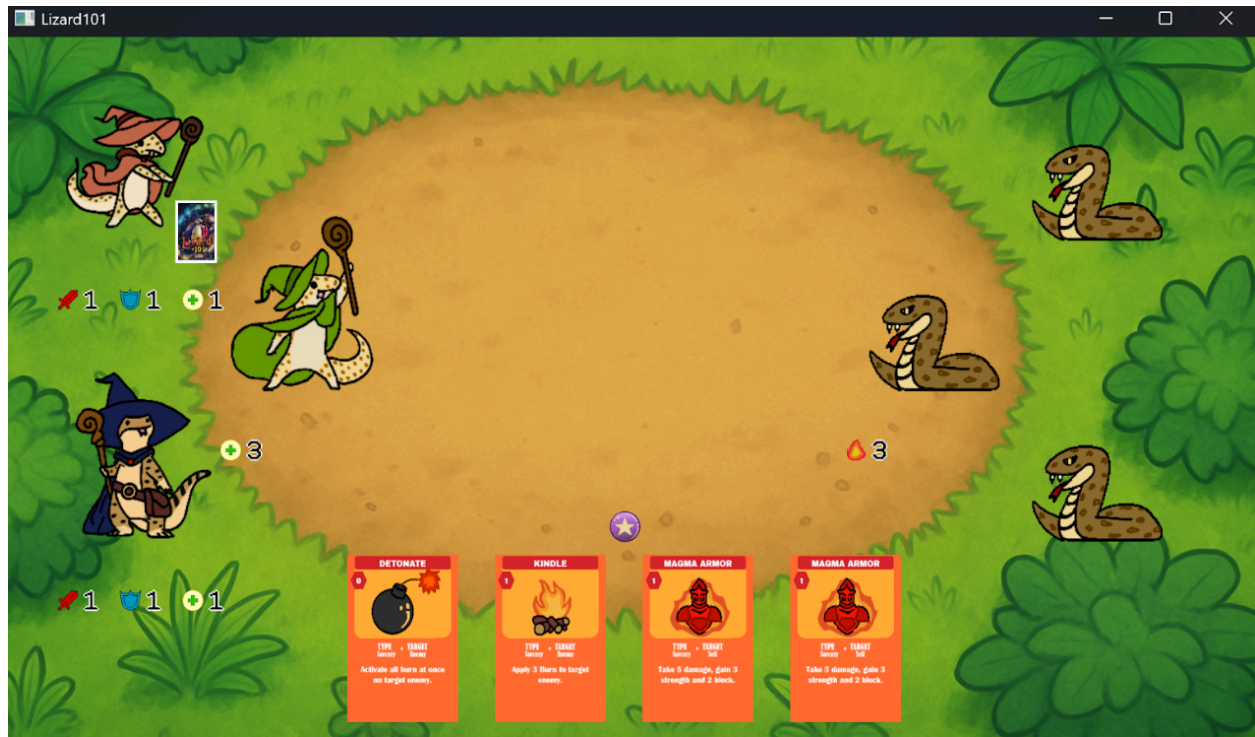


Figure 5 : Enemy game object creation using the ECS

```
for (int i = 0; i < numEnemies; ++i) {
    float x = baseX + offsetX[i];
    float y = baseY + offsetY[i];
    Entity* e = new Entity("EnemyEntity", x, y, actorW, actorH);
    e->setTag("ENEMY_" + std::to_string(i));
    e->addComponent<TextureRenderer>(
        renderer_,
        enemySprites[i]
    );
    e->addComponent<BoxCollider>();
    manager_.addEntity(e);
    enemyEntities_.push_back(e);
}
```

Figure 6: Input events being raised and used for mouse events

```

auto raiseDrag = [&](DragInfo::Phase phase, float x, float y) {
    eventManager_.raise(Event{
        EventType::Input, EventPriority::High, 0.0f,
        EventPayload{ DragInfo{ phase, x, y } }
    });
};

if (curMouseL && !prevMouseL) {
    raiseDrag(DragInfo::Phase::Start, mx, my);
}
else if (curMouseL && prevMouseL) {
    raiseDrag(DragInfo::Phase::Move, mx, my);
}
else if (!curMouseL && prevMouseL) {
    raiseDrag(DragInfo::Phase::End, mx, my);
}
prevMouseL = curMouseL;

```